

FE Bar Submission

Automated Call Summarization on Databricks

Github Repository: https://github.com/xintia-gyenge_data/salt-bank-voicescribe

1. Business Problem

Salt Bank operators handle customer calls all day. Today, after every call, each operator must manually write a summary of the conversation and file it into an internal tool (e.g., Jira). This is slow, inconsistent, error-prone, and does not scale — it consumes agent time that could be spent helping customers, and summary quality varies by person.

VoiceScribe automates this end to end: the call is captured, transcribed with speech-to-text, summarized and structured by AI (language-independent — Romanian and English), and stored automatically in a searchable internal application, with analytics for supervisors. No manual writing, consistent quality, instant filing.

2. Solution Overview

- **Capture:** A phone call is recorded (Twilio Voice in production; synthetic pre-generated recordings in the repo).
- **Transcribe:** Speech-to-text (Whisper) turns audio into a transcript, in the call's original language.
- **Summarize & structure:** A GenAI agent produces a concise summary plus structured fields — category, sentiment, action items, follow-up ticket — independent of language.
- **Store & serve:** Results land in the lakehouse (governed by Unity Catalog) and are served to operators through a Databricks App backed by Lakebase.
- **Analyze:** Supervisors query call trends in natural language via Genie.

3. Architecture — The Six Layers

Layer 1 — Lakeflow + Unity Catalog (ingestion & medallion)

- A Lakeflow Declarative Pipeline implements the Bronze → Silver → Gold medallion.
- **Bronze:** raw call audio stored in a Unity Catalog Volume + a call-metadata table (call id, from/to, agent, timestamp, duration, language).
- **Silver:** transcripts (output of the STT model), one row per call.
- **Gold:** structured summaries and enrichments (summary text, category, sentiment, action items, detected language, linked ticket id).

- Unity Catalog governs every asset — tables, volumes, models, functions — with lineage and fine-grained access control. This is the governance backbone.

Layer 2 — Lakebase (operational store)

- A Lakebase (managed Postgres / OLTP) database serves the internal application with low-latency reads and writes.
- Holds the operational records the app needs: summary records, ticket/approval status, agent assignments, edit history.
- Kept in sync with the lakehouse Gold tables so operational and analytical views agree.

Layer 3 — ML (speech-to-text)

- A Whisper speech-to-text model deployed on Databricks Model Serving performs language-independent transcription (Romanian & English).
- Optional lightweight classifier for call category / sentiment as a classical ML component.

Layer 4 — GenAI Agent (summarization & action extraction)

- An agent deployed on Model Serving takes a transcript and returns a structured, language-independent summary plus extracted action items, category, and sentiment.
- The agent can call a Unity Catalog function tool to create a Jira-style follow-up ticket — directly automating the manual step operators do today.

Layer 5 — Genie (natural-language analytics)

- A Genie space over the Gold analytics tables lets supervisors ask questions in plain language: "How many fraud-related calls last week?", "Average handle time by agent?", "Top 5 call reasons this month?".

Layer 6 — Databricks App (internal site)

- A React + FastAPI Databricks App is the operator-facing internal site.
- **Operators:** play the audio, read the transcript and AI summary, edit/approve the summary, search past calls, and file the ticket with one click.
- **Supervisors:** a dashboard with call metrics and an embedded Genie experience.

4. Twilio Integration (realistic test calls)

The Twilio path is built fully so the solution can be tested with real calls later:

- Twilio Voice records a call and sends a webhook (recording URL + call metadata: call SID, from/to, duration, agent, timestamp) to an ingest route in the app / a Databricks Job.

- The ingest route downloads the recording into the Bronze UC Volume and inserts the metadata row, triggering the pipeline.
- Because the customer has no Twilio account yet, the integration is pluggable: a real Twilio adapter and a synthetic-audio fallback (pre-generated text-to-speech call recordings in Romanian and English). The repository ships with the synthetic fallback only — no real customer or audio data — satisfying the submission's data rule.

5. Data Flow (summary)

1. Call recorded → webhook → ingest route.
2. Audio saved to Bronze UC Volume; metadata row inserted.
3. Lakeflow pipeline: STT (Layer 3) → transcript (Silver).
4. GenAI agent (Layer 4) → structured summary (Gold) + ticket via UC function.
5. Gold synced to Lakebase (Layer 2).
6. Operators review/approve in the App (Layer 6); supervisors analyze via Genie (Layer 5).
7. Unity Catalog (Layer 1) governs and traces the entire flow.

6. Languages

The solution demonstrates language-independent processing for Romanian and English: transcription detects and preserves the spoken language, and the summary/structuring is produced consistently regardless of the input language.

7. Screenshots

Unity Catalog Setup:

Catalog

Focus: ON

Shared Endpoint Serverless XL

Catalog

salt

My organization

salt_bank_voicescribe

voicescribe

Tables

gold_call_summaries

silver_transcripts

bronze_calls

Volumes

raw_audio

Functions

mask_phone

create_ticket

redact_pii_text

voicescribe_pipeline

Tables

silver_transcripts

gold_call_summaries

bronze_transcripts

bronze_calls

Volumes

landing

Functions

mask_phone

create_ticket

redact_pii_text

default

information_schema

Catalog Explorer

salt_bank_voicescribe

See compute details for more information.

Share

Create schema

Overview

Details

Permissions

Policies

Workspaces

Description

Salt Bank VoiceScribe demo – synthetic data only

Filter schemas 4 schemas

Name	Owner	Created at
default	xintia.gyenge@databricks.com	Sep 20, 2026, 08:50 PM
information_schema	System user	Sep 20, 2026, 08:50 PM
voicescribe	xintia.gyenge@databricks.com	Sep 20, 2026, 08:50 PM
voicescribe_pipeline	xintia.gyenge@databricks.com	Sep 20, 2026, 10:06 PM

About this catalog

Owner Xintia Gyenge

Tags

Add tags

Policies

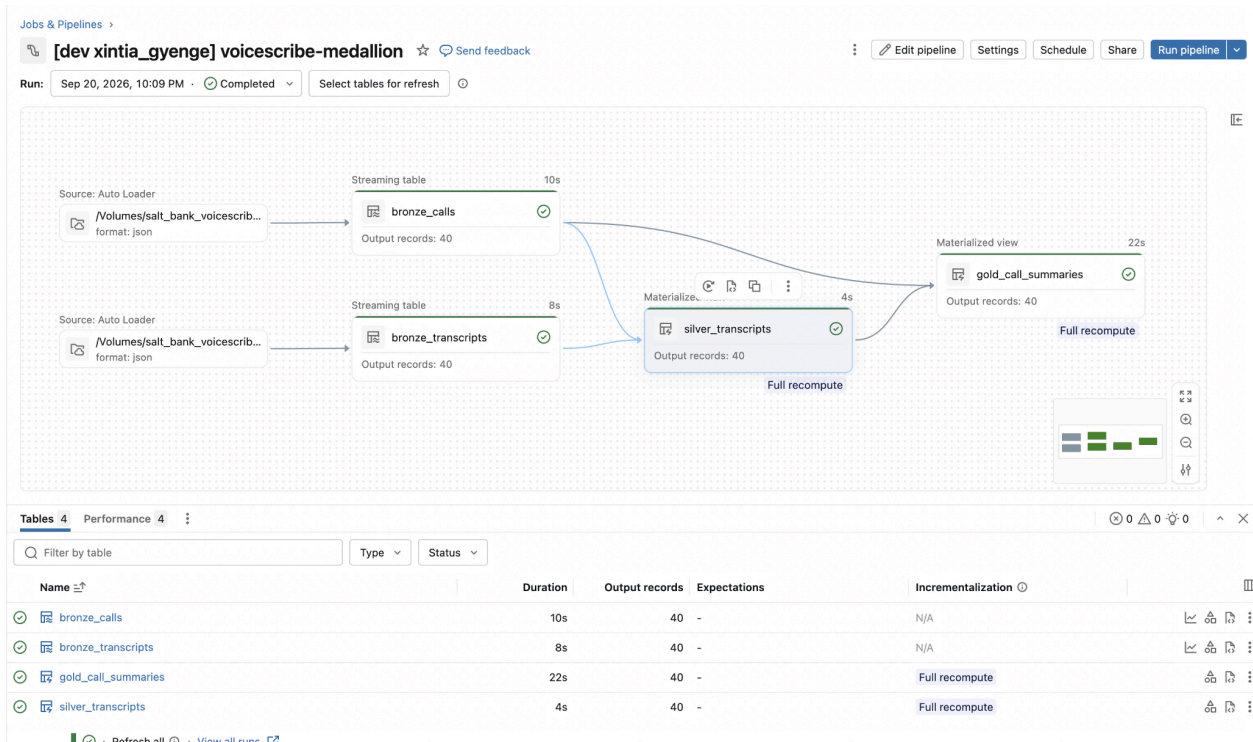
New policy

Data Classification

Disabled

Enable

Lakeflow pipeline run:



Model Serving Endpoint:

Serving endpoints >

voicescribe-whisper-stt

PermissionsEditQuery endpoint

OverviewMetricsDeploymentsLogs

Readyhttps://adb-984752964297111.11.azuredatabricks.net/serving-endpoints/voicescribe-whisper-stt/invocations

AI GatewayConfigure AI Gateway

Usage monitoring: Not configuredRate limits: Not configuredInference tables and telemetry: Not configured

Active configuration

Entity	Version	Name	State	Compute	Traffic (%)
system.ai.whisper_large_v3	Version 3	whisper-stt	Ready	GPU Large (A100), Small 0-4 concurrency	100

View starter code

cURLPython (JSON)Python (Protobuf)SQL

```
curl \
-u token:$DATABRICKS_TOKEN \
-X POST \
-H "Content-Type: application/json" \
-d@data.json \
https://adb-984752964297111.11.azuredatabricks.net/serving-endpoints/voicescribe-whisper-stt/invocations
```

About this endpoint

Created by Xintia GyengeRoute optimization Not enabled

Tagslayer: 3-stt project: voicescribe

Serverless usage policy

DescriptionNo description

Get insights from Genie CodeUse Genie Code to help understand and troubleshoot your endpoint.Explain configuration

Serving endpoints >

voicescribe-whisper-stt

PermissionsEditQuery endpoint

OverviewMetricsDeploymentsLogs

Configure alerts

TimestampMessage

Sep 21, 2026, 12:54 PMEndpoint 'voicescribe-whisper-stt' entered READY state.

Sep 21, 2026, 12:54 PMEndpoint update succeeded for endpoint 'voicescribe-whisper-stt', config version 1.

Sep 21, 2026, 12:54 PMServed entity creation succeeded for served entity 'whisper-stt', config version 1.

Sep 21, 2026, 12:54 PMServed entity 'whisper-stt' entered DEPLOYMENT_READY state.

Sep 21, 2026, 12:53 PMServed entity 'whisper-stt' entered DEPLOYMENT_CREATING state: Deploying

Sep 21, 2026, 12:52 PMServed entity 'whisper-stt' entered DEPLOYMENT_CREATING state: Provisioning resources

Sep 21, 2026, 12:52 PMServed entity created for served entity 'whisper-stt', config version 1.

Sep 21, 2026, 12:52 PMEndpoint updated by xintia.gyenge@databricks.com.

Sep 21, 2026, 12:52 PMEndpoint created by xintia.gyenge@databricks.com.